

函数调用和格式化

- **函数调用：位置实参和关键字实参**
- 格式化（2.5.3和9.7）
 - 格式化运算符%（不介绍）
 - 字符串的格式化方法
 - f-string

函数调用

- 在函数调用时，首先要按照顺序计算各个参数的值后，才进行实际的函数调用

```
def cubic(n):  
    return n**3  
  
n = 3  
result = cubic(n)  
result = cubic(cubic(n))  
result = cubic(cubic(cubic(n)))
```

阅读程序写输出：

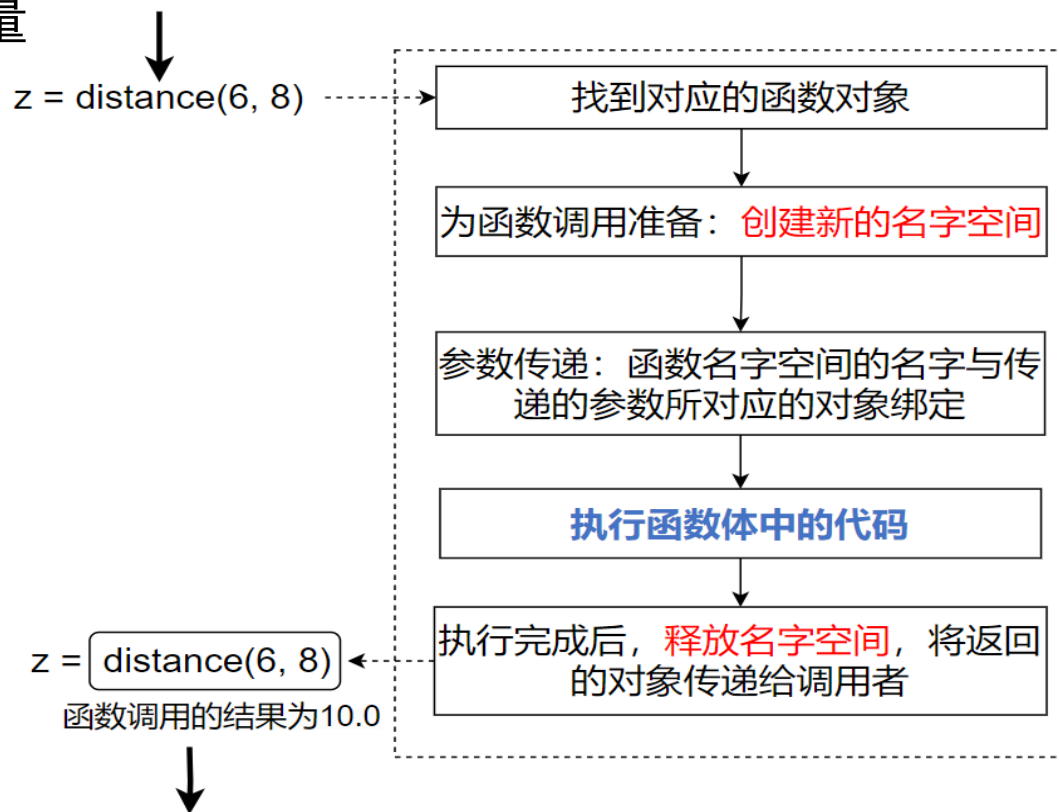
```
text = input(input('PS1= ') + '> ')
```

函数调用：参数传递

参数传递：实参(Actual Argument) 与形参(Formal Parameter)对应，传递实参给出的对象到相应形参

- 采用**赋值传递(pass by assignment)**的策略：本地名字空间的形参变量=调用者传递的实参对象
- 为了避免side effect，函数调用时为本次调用**创建一个新的名字空间（本地名字空间）**
 - 形参变量为本地名字空间的**局部变量**，其使用范围为当前函数的函数体
 - 函数体内**赋值类语句**引入的名字一般也是局部变量

```
def distance(x, y):  
    """返回直角坐标系中点(x, y)到原点的距离  
    传递的参数：x, y为点的X轴和Y轴坐标  
    """  
    x2 = x * x  
    y2 = y * y  
    return (x2 + y2) ** 0.5  
  
z = distance(6, 8)
```



函数调用的参数：位置和关键字参数（实参）

- 位置(positional) 实参: 基于位置传递值，按照参数出现的顺序逐个与函数定义时的参数(形参)对应
- 关键字实参: 基于参数名字传递值，格式name=expr，表示名为name的形参对应的是表达式expr求解后的对象
 - 关键字实参传递的顺序可以和形参顺序不一致
 - 关键字参数右边不能有位置参数，即调用时参数的顺序为位置参数(如果有)、关键字参数

```
def func1(a, b, c, d):  
    print(a, b, c, d)
```

```
func1(1, 2, 3, 4)
```

```
func1(a=1, b=2, c=3, d=4)
```

```
func1(b=2, a=1, d=4, c=3)
```

```
func1(1, 2, c=3, d=4)
```

```
func1(1, 2, 3)
```

```
# TypeError: func1() missing 1 required positional argument: 'd'
```

```
func1(1, 2, 3, 4, 5)
```

```
# TypeError: func1() takes 4 positional arguments but 5 were given
```

```
func1(1, 2, c=3, 5)
```

```
# SyntaxError: positional argument follows keyword argument
```

内置函数print

`print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)`

支持可变长的参数，将多个参数转换为字符串并且输出

- 在输出时，这些值之间以`sep`分隔；`sep`缺省为空格
- 所有值输出完之后，输出`end`参数，`end`缺省为换行
- 缺省输出到标准输出(屏幕)，也可通过`file`参数指定输出到相应的文件中
- 返回None

```
# 附加Hello, world到文件mytest.txt
fp = open('mytest.txt', 'a+')
print('Hello,world!', file=fp)
fp.close()
```

```
x, y, z='Mike', 4, 3.14
print(x, y, z)                #输出Mike 4 3.14
print(x, y, z, sep=', ')     #输出Mike, 4, 3.14
print(x, y, z, sep=', ', end='****') #输出Mike, 4, 3.14****
print(1, 2, 3, sep='')        #在上一行最后输出123
print(1, 2, 3, sep='+', end='=') #输出1+2+3=
print(1 + 2 + 3)              #在上一行最后输出6
print()                       #输出空行
```

主要内容

- 函数调用：位置实参和关键字实参

- **格式化**

- 格式化运算符%（不介绍）
- 字符串的格式化方法
- f-string

```
>>> 'Hi %s' % 'idle'
'Hi idle'
>>> 'x=%d, y=%5.2f' % (2, 2**3)
'x=2, y= 8.00'
```

格式化

字符串对象的format方法: `format_string.format(p0,p1,... k0=v0,k1=v1,...)`

一个或多个对象按format_string给出的格式要求转换为字符串

- 格式化字符串(format_string)是格式化的模板, 由固定文本以及替代字段(replacement field)混合而成
 - 格式化的结果: 固定文本原封不动; 替代字段被某个值格式化后的字符串所替换
 - 替代字段: 一对大括号包含, 给出了**如何格式化某个值** `{[fieldname][!conversion][:formatspec]}`
 - 可选的fieldname: 哪个值
 - 要格式化的值通过函数调用时的参数传递完成
 - 可通过位置参数以及关键字参数传递
 - 可选的.attr或[index]: 值的指定属性或者值的对应位置 (下标形式) 的元素
 - 可选的!conversion: 该值进行转换后再格式化
 - !r: 调用repr(value)
 - !s: 调用str(value)
 - !a: 调用ascii(value)
 - 可选的:formatspec: 怎么格式化
 - 如果省略, 表示采用要格式化的值所支持的**缺省格式化方法**。大部分情况下调用str转换为字符

```
>>> "First {0}, Second {1:d}, Third {2:5.2f}".format('idle',  
90, 93.4)  
'First idle, Second 90, Third 93.40'
```

格式化：哪个值？

替代字段：一对大括号包含，给出了**如何格式化某个值** `{[fieldname][!conversion][:formatspec]}`

- 可选的fieldname：哪个值
 - 要格式化的值通过函数调用时的参数传递完成：可通过位置参数以及关键字参数传递
 - 整数：哪个位置参数。**位置参数编号从0开始**
 - 名字：哪个关键字参数
 - 如果省略，采用自动编号，下一个位置参数
 - **以左括号为基准自动编号**。第1个左括号对应的位置参数编号为0，下一个为1...
 - **自动编号和人为编号不能混合使用**。比如"`{ } {1}`".format(1, 2)会报错
 - 可选的.attr或[index]：值的指定属性或者值的对应位置（下标形式）的元素
- 可选的!conversion：该值进行转换后再格式化，可取!r, !a, !s
- 可选的:formatspec：怎么格式化
- 如果要输出{或者}，使用两个对应的大括号，即**{{或者}}**

替代字段的四种可能：

`{2:5.2f}` `{2}`
`{:5.2f}` `{}`

```
>>> "First {0}, Second {1:d}, Third {2:5.2f}".format('idle', 90, 93.4)
'First idle, Second 90, Third 93.40'
>>> "First {}, Second {}, Third {:5.2f}".format('idle', 90, 93.4) # 自动编号
'First idle, Second 90, Third 93.40'
```

格式化：哪个值？

- 替代字段：一对大括号包含，给出了**如何格式化某个值** `{[fieldname][!conversion][:formatspec]}`
- 可选的fieldname：哪个值，哪个位置参数(数值或省略)或关键字参数(名字)
 - **位置参数编号从0开始**
 - 自动编号时以**左括号为准自动编号。自动编号和人为编号不能混合使用**
 - 可选的.attr或[index]：值的指定属性或者值的对应位置（下标形式）的元素
- 如果要输出{或者}，使用两个对应的大括号，即**{{或者}}**

```
>>> "{0:6.2f} or {0:6.3f}".format(3.1415) # 不需要重复传递多个值
' 3.14 or  3.142'
>>> "Art: {a:5d}, Price: {p:8.2f}".format(a=453, p=59.058) # 关键字参数
'Art:   453, Price:   59.06'
>>> "Name: {:6} Art: {a:5d}, Price: {p:8.2f}".format('Art1', a=453, p=59.058) # 自动编号 + 关键字参数
'Name: Art1  Art:   453, Price:   59.06'
>>> print("Second {1}, First {0}, Third {2}\nArt: {a:5d}, Price: {p:8.2f}".format('idle', 90, 93.4,
a=453, p=59.058 )) # 人为编号的位置参数 + 关键字参数
Second 90, First idle, Third 93.4
Art:   453, Price:   59.06

>>> '{{{}} > {}}' .format(54, 42, 54 > 42) # {{替代以 {
'{54 > 42} True'
```

格式化：哪个值？

- 替代字段：一对大括号包含，给出了**如何格式化某个值** `{[fieldname][!conversion][:formatspec]}`
- 可选的fieldname：哪个值
 - 要格式化的值通过函数调用时的参数传递完成：可通过位置参数以及关键字参数传递
 - 可选的.attr或[index]：值的指定属性或者值的对应位置（下标形式）的元素
- 可选的!conversion：该值转换后再格式化，可取!r, !a, !s，分别调用repr/ascii/str函数后再格式化
- 可选的:formatspec：怎么格式化
- 如果要输出{或者}，使用两个对应的大括号，即**{{或者}}**

```
>>> c = 3-5j
>>> '{0}, 实部 {0.real} 虚部 {0.imag}.'.format(c)
'(3-5j), 实部 3.0 虚部 -5.0.'
>>> coord = (3, 5)
>>> 'X: {0[0]}; Y: {0[1]}'.format(coord)
'X: 3; Y: 5'
>>> text = '{0}\n{0!r}\n{0!a}'.format('python程序设计')
>>> print(text)
python程序设计
'python程序设计'
'python\u7a0b\u5e8f\u8bbe\u8ba1'
```

格式化：怎么格式化？

替代字段：一对大括号包含，给出了**如何格式化某个值** `{[fieldname][!conversion][:formatspec]}`

- 可选的:formatspec：怎么格式化 `[[fill]align][sign][#][0][width][,或_][.precision] [,或_][typecode]`
- 最小宽度：实际的值格式化后的宽度不够时填充到指定的宽度
 - 如果值实际宽度超过了最小宽度时，也不会截取，而是允许超过最小宽度
 - 对齐方式：
 - **< 左对齐** 大部分对象缺省为左对齐
 - **> 右对齐** 数值类型缺省为右对齐
 - **^ 中间对齐**
 - **=** 用于数值类型，右对齐，<符号(如果有)> <填充字符> <数值>，比如 `'- 45'`
 - 填充字符：默认为空格
 - 对齐字符之前有字符(fill)，填充字符为指定的字符
 - 0表示填充0。同时指定了fill时，fill优先
- 精度：一般用于数值类型的浮点数表示，舍入时四舍六入五成双（银行家法则）
 - typecode为f, e或E时，小数点后的位数。默认为6
 - .0, 小数点后0位，即不包含小数部分
 - typecode为g或G时，表示有效数字个数，缺省6
- 符号：+, -或空格。+表示总是包含符号；空格表示负数有符号，正数时为空格；**-表示仅负数有符号(缺省)**

格式化：怎么格式化？

- 可选的:format)spec: 怎么格式化 `[[fill]align][sign][#][0][width][,或_][.precision [,或_]][typecode]`
- 最小宽度, 精度, 符号
- **# 替代格式**, 用于数值类型。typecode为b/o/x/X时, 加上前缀0b/0o/0x/0X
- 分组: 用于数值类型, 可取值**, 或_**。python3.14允许分组出现在精度部分
 - typecode为b/o/x/X时, `_`表示从右边开始每4个数字加下划线
 - typecode为d/f/e/g时, 整数部分或者小数点之后的部分每3个数字加逗号或下划线
- 格式字符(typecode): 单字符, 表示对应的值采用对应的方式格式化
 - s: 对应值为字符串类型
 - d: 对应值为整数, 格式化为十进制整数字符串
 - o: 对应值为整数, 格式化为八进制整数字符串
 - x或X: 对应值为整数, 格式化为十六进制整数字符串, x表示采用大写形式
 - f: 对应值为数值, 格式化为十进制小数表示的字符串, 精度默认为6
 - e: 对应值为数值, 格式化为科学计数法表示的字符串, 精度默认为6。E表示大写形式
 - g或G: 对应值为数值, 格式化为浮点数(包括整数) 或科学计数法, 有效数字默认为6。G表示大写形式
 - %: 百分比, 比如`{:.2%}'.format(0.456)`的结果为`'45.60%'`
 - 不指定typecode时: 整数值默认为d, 浮点数值默认为g, 字符串默认为s
 - 可自定义typecode, 该对象的类型中实现`__format__(self, format_spec)`, 默认实现调用`str(value)`

`help('FORMATTING')` 查看完整的格式化描述

格式化示例

```
>>> '[{:s}]'.format('python')
'[python]'
# 值宽度已经超过最小宽度
>>> '[{:3}]'.format('python')
'[python]'
>>> '[{:10}]'.format('python')
'[python      ]'
>>> '[{:>10s}]'.format('python')
'[      python]'
>>> '[{:*^10}]'.format('python')
'[**python**]'
```

```
>>> '[{:10d}]'.format(12345)
'[      12345]'
>>> '[{:10}]'.format(-12345)
'[-12345]'
# 正数也要符号, 右对齐, 填充字符在符号之后
>>> '[{:+=10d}]'.format(12345)
'[+      12345]'
# 正数时符号位为空格, 填充0
>>> '[{: 010d}]'.format(12345)
'[ 000012345]'
>>> '[{:<10d}]'.format(12345)
'[12345      ]'
>>> '[{:10b}]'.format(12345)
'[11000000111001]'
>>> '[{:#10o}]'.format(12345)
'[    0o30071]'
>>> '[{:*^#10X}]'.format(12345678)
'[*0XBC614E*]'
>>> '[{: ,}]'.format(12345678)
'[12,345,678]'
>>> '[{:#_o}]'.format(123456789)
'[0o7_2674_6425]'
```

格式化示例

默认小数点后6位

```
>>> '{:f}'.format(12345)
```

```
'[12345.000000]'
```

```
>>> '{:10.2f}'.format(12345)
```

```
'[ 12345.00]'
```

四舍六入五成双

```
>>> '{:10.2f}'.format(12345.6789)
```

```
'[ 12345.68]'
```

仅仅包含整数部分

```
>>> '{:<10.0f}'.format(12345.6789)
```

```
'[12346      ]'
```

```
>>> '{:10.0f}'.format(12345.5)
```

```
'[      12346]'
```

```
>>> '{:10.0f}'.format(12344.5)
```

```
'[      12344]'
```

科学计数法,尾数在[1,10)

```
>>> '{:10e}'.format(12345.6789)
```

```
'[1.234568e+04]'
```

```
>>> '{:10.2e}'.format(12345.6789)
```

```
'[ 1.23e+04]'
```

有效数字默认为6

```
>>> '{:10g}'.format(12345.6789)
```

```
'[      12345.7]'
```

```
>>> '{:10.8g}'.format(12345.6789)
```

```
'[ 12345.679]'
```

```
>>> '{:10.2g}'.format(12345.6789)
```

```
'[      1.2e+04]'
```

格式化：示例

- 标题行和数据行可以指定**相同的宽度和对齐方式**

```
print('{:<15} {:<30s} {:<15s} {:s}'.format('ID', 'Name', 'Score', 'Memo'))  
print('{:<15} {:<30s} {:<15.2f} {:s}'.format('00001506', 'Guido van Rossum', 100, 'Founder of Python'))
```

ID	Name	Score	Memo
00001506	Guido van Rossum	100.00	Founder of Python

- 中英文混排时**，指定宽度对齐的方法可能会出现没有对齐的情况

```
>>> print('{:<15} {:<30s} {:<15s} {:s}'.format('ID', '姓名', '分数', '说明'))  
... print('{:<15} {:<30s} {:<15.2f} {:s}'.format('00001506', 'Guido van Rossum', 100, 'Founder of Python'))  
...
```

ID	姓名	分数	说明
00001506	Guido van Rossum	100.00	Founder of Python

格式化：格式化说明符动态构造

- 通过嵌套支持格式化说明符的动态构造，但仅允许嵌套一层

```
>>> '{0:{1}}'.format(12345, 10)    # '{0:10}'.format(12345)
'      12345'
```

```
>>> '{:{:s}{align}{width}}'.format('hello', '*', align='^', width=20)
'*****hello*****'
```

上述例子：通过自动编号和关键字参数传递参数

嵌套的部分解析完成后等价于 '{:*^20}'.format('hello')

字符串对象的format方法

字符串的format方法为**实例方法**

`format_string.format(p0,p1,... k0=v0,k1=v1,...)`

- 可看成(特殊的)函数对象，隐藏了的参数为字符串对象
- 可以暂时保存字符串的format方法到某个变量，即**该变量指向该函数对象**

```
>>> "Art: {a:5d}, Price: {p:8.2f}".format(a=453, p=59.058)
'Art:    453, Price:    59.06'

>>> format_func = "Art: {a:5d}, Price: {p:8.2f}".format
>>> format_func
<built-in method format of str object at 0x05861090>
>>> format_func(a=453, p=59.058)
'Art:    453, Price:    59.06'
```

内置函数format: 仅仅格式化单个值

`format(value, format_spec='')` 将value按照format_spec进行格式化

- 等价于 `'{:format_spec}'.format(value)`
- `format_spec`与前面字符串的format方法中的格式一致（注意不包括字段部分及冒号）

`[[fill]align][sign][#][0][width][,][.precision][typecode]`

`{{fieldname}[!conversion][:formatspec]}`

```
>>> '{:6.2f}'.format(3.1415)
' 3.14'
>>> format(3.1415, '6.2f')
' 3.14'
```

字符串的方法：字符串对象的填充和对齐（9.6.1）

- `strobj.center/ljust/rjust/zfill()`：指定宽度和填充字符。返回新字符串，原字符串**居中或左对齐或右对齐**，并使用**指定字符（默认空格）填充**，保证长度至少为`width`

方法	描述	等价于
<code>s.center(width[,fillchar])</code>	居中对齐	<code>'{: }^{}'.format(s, fillchar, width)</code>
<code>s.ljust(width[,fillchar])</code>	左对齐	<code>'{: }<{}'.format(s, fillchar, width)</code>
<code>s.rjust(width[,fillchar])</code>	右对齐	<code>'{: }>{}'.format(s, fillchar, width)</code>
<code>s.zfill(width)</code>	右对齐，填充0	<code>rjust(width, '0')</code> 或 <code>'{:0>{}'.format(s, width)</code>

- `expandtabs([tabsize])`: 将`\t`转换为多个(`tabsize`)空格，缺省为8个

```
>>> '1\t2\t3'.expandtabs()
'1      2      3'
>>> '1\t2\t3'.expandtabs(2)
'1 2 3'
```

```
>>> s = 'Hello world!'
>>> s.center(20)
'      Hello world!      '
>>> s.center(20, '=')
'====Hello world!===='
>>> '{:=^20s}'.format(s)
'====Hello world!===='
>>> format(s, '=^20s')
'====Hello world!===='
```

```
>>> s.ljust(20, '=')
'Hello world!======'
>>> '{:=<20s}'.format(s)
'Hello world!======'
```

```
>>> s.rjust(20, '=')
'====Hello world!'
>>> '{:=>20s}'.format(s)
'====Hello world!'
```

f-string(formatted string literal)

python3.6引入, 语法: f'.....{expression:format_spec}.....'

- 固定文本 + 表达式的值按照指定的方式进行格式化
- 与传统字符串定义类似, 可以采用单双引号以及三引号, 只是前面添加f或者F
- 大括号包括的部分被替代:
 - **表达式expression**运算后的结果进行格式化
 - **:format_spec** 指出怎么进行格式化。可省略, 表示采用缺省方式格式化
- {{表示{, }}表示}

fstring.py

```
name = 'Tony'
age = 50
print(f'My name is {name}, my age next year is {age+1}.')

print(f'{"Name":<15}{"Score":<15}')

import math
print(f'pi={math.pi:.5f}, log(2)={math.log(2,10):.5f}')
```

My name is Tony, my age next year is 51.

Name Score

pi=3.14159, log(2)=0.30103

f-string(formatted string literal)

python3.8, {expression:format_spec} 支持 {expression = :format_spec}

- 格式化的结果为 expression = <value>
- 表达式本身出现的（包括=前后的）空格也保留

```
>>> f'{2**16 - 1 = }'  
'2**16 - 1 = 65535'  
>>> f'{2**16 = : ,}'  
'2**16 = 65,536'
```

【拓展】python3.12, fstring中的表达式:

- 可以采用相同的字符串界定符（引号）
- 可以使用转义字符

```
text = f'new feature introduced by python3.12\n[{\'\\\''.join(('f1', 'f2', 'f3')):^10}]'  
print(text)
```

```
new feature introduced by python3.12  
[ f1\f2\f3 ]
```

额外的Slides

字符串对象支持的算术运算符：%

format_string % value

format_string %map (不介绍)

format_string % (value1, value2, ...)

一个或多个对象按format_string给出的格式要求转换为字符串。产生类似C语言的printf格式化输出

- 格式化字符串(format_string)是格式化的模板，由固定文本以及格式说明符混合而成
 - 格式说明符：**%[flags][width][.precision]type**
 - 说明后面的值如何转换为字符串
 - 如果有多个格式说明符，则采用第2种格式：
 - %后面的对象是元组 (tuple) 对象，它是由多个值组成的容器对象
 - 在格式化时按照位置顺序与前面的格式说明符对应
- 格式化运算后的结果：
 - 固定文本原封不动
 - 格式说明符部分被一个字符串替换，该字符串由运算符%后面的对应位置的值根据相应的格式说明转换而来

```
>>> 'Hi %s' % 'idle'
'Hi idle'
>>> 'x=%d, y=%5.2f' % (2, 2**3)
'x=2, y= 8.00'
```

格式说明符

%[-] [0] [+] [#][width] [.precision] type

格式说明符: % [flags] [width] [.precision] type

- 格式字符type: 要求与待格式化的值类型匹配

```
>>> 'Hi %s' % 'idle'
'Hi idle'
>>> 'x=%d, y=%5.2f' % (2, 2**3)
'x=2, y= 8.00'
>>> '%d%%' % 30.5
'30%'
```

格式字符	说明	格式字符	说明
%s	任意对象obj, 输出str(obj)	%x或%X	整数, 输出十六进制字符串, x表示大写
%r或%a	与%s类似, 只是为repr(obj)或ascii(obj)	%e	数值, 输出科学计数法表示, 基底为e, 精度缺省为6
%c	整数n或字符。输出对应的字符。对于整数n, 首先调用chr(n)得到字符	%E	与%e类似, 只是基底为E
%d	数值, 首先转换为整数, 再转换为十进制字符串	%f、%F	数值, 输出小数点表示法, 精度缺省为6
%i	与%d等价, 输出十进制整数字符串	%g或%G	科学计数法或浮点数(归一化后指数小于-4或大于等于精度即有效数字个数时, 采用科学计数法)
%o	整数, 输出八进制字符串	%%	字符%

- %s %r %a对应的值可是任意类型
- %c 对应的值为整数或字符, %o %x对应值为整数
- 其他格式字符要求对应的值为数值类型

```
>>> "%d" % "555"
Traceback (most recent call last):
TypeError: %d format: a number is required, not str
```

格式说明符

%[-] [0] [+] [#][width] [.precision] type

格式说明符: % [flags] [width] [.precision] type

- 宽度 (可选): 最小宽度。如果实际的值格式化后的宽度不够时填充
 - 如果值实际宽度超过了最小宽度时, 也不会截取, 而是允许超过最小宽度
- 精度 (可选): 四舍六入五成双 (银行家法则)
 - type为f, 小数点后的位数, .相当于.0, 即小数点后0位 (不包含小数部分)。缺省为6。
 - type为g时, 表示有效数字个数, 缺省6
- flags: 可选, 表示对齐、填充、是否添加前缀或符号等, 格式为 [-][0][+][#]
 - 默认右对齐。-表示左对齐
 - 只有指定了最小宽度, 且实际的值格式化后的宽度不够时, 对齐和填充才有意义
 - 填充: 默认填充空格, 0表示填充0
 - 符号: 正数缺省省略符号。如果为+表示正数也要添加符号; 如果为空格, 对于正数添加空格而不是符号
 - #: type为xXo等(转换为十六进和八进制)时前面加上相应的进制表示前缀(0x, 0X或0o等)

```
>>> '%s' % 123.4
'123.4'
>>> '%8s' % 123.4
'    123.4'
>>> '%-8s' % 123.4
'123.4    '
```

```
>>> '%f' % 123.4
'123.400000'
>>> '%.2f' % 123.4
'123.40'
>>> '%4.2f' % 123.4
'123.40'
>>> '%8.2f' % 123.4
'    123.40'
>>> '%-8.2f' % 123.4
'123.40    '
```

```
>>> '%8.f' % 123.4
'    123'
>>> '%08.2f' % 123.4
'00123.40'
>>> '%+8.2f' % 123.4
' +123.40'
>>> '%e' % 123.4
'1.234000e+02'
>>> '%E' % 123.4
'1.234000E+02'
```

```
>>> '%d' % 123
'123'
>>> '%8d' % 123
'    123'
>>> '%-8d' % 123
'123    '
>>> '%08d' % 123
'00000123'
```

```
>>> '%o' % 123
'173'
>>> '%#o' % 123
'0o173'
>>> '%x' % 123
'7b'
>>> '%#x' % 123
'0x7b'
>>> '%#X' % 123
'0X7B'
```

格式说明符 【拓展内容】

% [flags] [width] [.precision] type

%g或%G: 数值，采用科学计数法或浮点数（包括整数）表示

- 基于归一化后的指数决定
- 如果指数小于-4（足够小）或大于等于精度即有效数字个数（足够大）时，采用科学计数法
- 精度指的是有效数字个数，缺省6
 - 有效数字从第1个非零数字开始计算（前导0不计入）
- 去除冗余0：自动删除末尾无意义的零。比如123.450输出为123.45，而123.0输出123

```
>>> '%g' % 0.0001234567
'0.000123457'
>>> '%g' % 0.00001234567
'1.23457e-05'
>>> '%.3g' % 0.00001234567
'1.23e-05'
```

```
>>> '%g' % 123456789
'1.23457e+08'
>>> '%g' % 123456
'123456'
>>> '%.5g' % 123456
'1.2346e+05'
```

最小宽度和最小精度的动态构造【拓展内容】

- 最小宽度和精度如果要**动态构造**，而不是写在格式化字符串中时，可以用*代替，表示具体的取值为后面的元组中的下一个值

```
>>> '%10.*f' % (2, 3.1415)
'          3.14'
>>> '%*.*f' % (10, 2, 3.1415)
'          3.14'
```

'%10.*f' 最小精度的位置为*，来自于下一个值2

- 最小宽度的位置为*，来自于下一个值10
- 最小精度的位置为*，来自于下一个值2

字符串格式化

- 标题行和数据行可以指定**相同的宽度和对齐方式**

```
print('%-15s %-30s %-15s %s' % ('ID', 'Name', 'Score', 'Memo'))  
print('%-15s %-30s %-15.2f %s' % ('00001506', 'Guido van Rossum', 100, 'Founder of Python'))
```

ID	Name	Score	Memo
00001506	Guido van Rossum	100.00	Founder of Python

- format_str % value, **如果value包含了其他运算符时需要注意运算的优先级顺序**
 - * % / // 具有相同的优先级，优先级相同时采用从左到右的顺序计算

```
>>> '%d' % 12 * 4
```

```
'12121212'
```

```
>>> '%d' % 12 + 4
```

```
Traceback (most recent call last):
```

```
File "<pyshell#0>", line 1, in <module>
```

```
'%d'%12 + 4
```

```
TypeError: can only concatenate str (not "int") to str
```

```
>>> '%d' % (12 * 4)
```

```
'48'
```

%和*的优先级一样，
采用左结合

格式化输出：%运算符的局限

目前为止已经了解的格式化输出方法

- `print(obj1, obj2, ..., sep=' ', end='\n')` 通过`sep`和`end`进行格式化
- `print(str(obj1) + sep + str(obj2) ...)` 利用`+`运算符**组合成一个大的字符串**后输出
- `print("Art: %5d, Price: %8.2f" % (453, 59.058))` 利用**%运算符**进行格式化
- %运算符的局限
 - 格式化字符串中的格式说明符与后面的值**按照位置**进行对应，不是非常方便
 - 某个值要格式化多次时要重复多次
 - 增加或删除格式说明符时，或调整先后顺序时，也要相应地调整后面的值的顺序
 - 只能支持右对齐和左对齐，且填充字符只能是空格（对于`%d` `%f`等数值类型说明符还可填充0）
 - 除了`%s/%d/%f/%e/%g`等，无法增加新的类型的格式说明符

`format_string % value`

`format_string % (value1, value2, ...)`

`format_string % map` 与字典对象结合，不展开

字符串对象的format方法

语法	<code>format_string % value</code> <code>format_string % (value1, value2, ...)</code> <code>format_string %map (不介绍)</code>	<code>format_string.format(p0,p1,... k0=v0,k1=v1,...)</code>
格式说明	<code>%[(key)][flags][width][.precision]typecode</code>	<code>{[fieldname][!conversion][:formatspec]}</code>
格式化的值	按照位置匹配 (不介绍) (key)表示字典对象map中key对应的值	• <code>fieldname</code>为数值, 第几个(从0开始) 位置参数 • <code>fieldname</code>为名字, 表示关键字参数 • <code>fieldname.attr</code>: 表示对应的属性 • <code>fieldname[idx]</code>:表示对应的下标 • 无<code>fieldname</code>表示自动编号, 下一个位置参数
如何格式化	<code>[-] [0] [+] [#][width] [.precision] typecode</code> typecode为s/r/a, 对应任意类型值 typecode为d/i/e/f/g, 对应数值类型值 typecode为x/o, 对应整数类型值	<code>[[fill]align][sign][#][0][width][,或_][.precision][typecode]</code> • typecode省略, 按照对应的值的缺省方法格式化 • s, 对应值必须为字符串类型 • 新增b, 格式化为二进制整数字符串 • 新增%, 格式化为百分比
填充和对齐	• 缺省为右对齐, -表示左对齐 • 缺省填充空格, 0表示填充0 (要求为数值类型)	• 数值类型的值缺省右对齐, 其他类型值缺省左对齐 • align为<>^, 对应左、右和居中对齐 • align字符前如果有字符, 表示填充该字符 • 0表示填充0, fill字符优先
千位分割	不支持	,或_ 对于数值类型的值, 其整数部分每千位以逗号或下划线隔开